



What scale changed

30 minutes

BY THE END OF THIS CAPSULE YOU WILL BE ABLE TO

Explain what a foundation model actually is, and why some of its failures are structural rather than random.



A model is two files

That is genuinely all it is. Everything else is how those numbers were produced.

parameters

140 GB

70 billion numbers.
Each one 2 bytes.
That is the model.

run.c

~500 lines

The program that reads them.
No internet needed.
Runs on a good laptop.

Compiled together, these two files need no network connection at all.

↳ 70B params × 2 bytes = 140GB because weights are stored at float16 precision. Quantise to int8 or int4 and the same model fits in 70GB or 35GB — with some quality loss. This is why models run on phones.

Example: Llama 2 70B · Karpathy, Intro to Large Language Models (2023)



Where those numbers came from

Training is a one-time compression. Expensive, slow, and done rarely.

10 TB

of text scraped from the internet

6,000

GPUs running in parallel

12 days

of continuous training

~\$2M

and that is a mid-size model

10 TB in, 140 GB out — roughly 100× compression, and lossy. The model does not store the text. It stores a generalised impression of it. Karpathy calls it “a zip file of the internet”, and the lossiness is exactly where hallucination comes from.

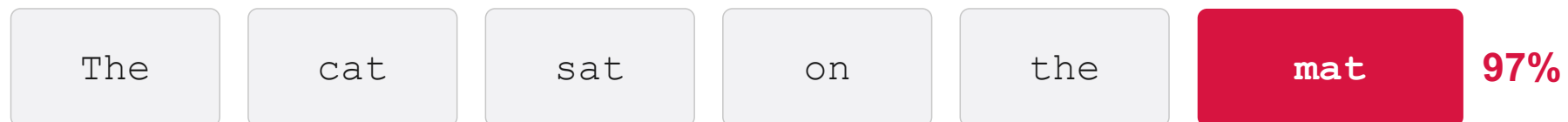
↳ Frontier models are 10–100× this. Note the asymmetry: training is a huge one-off capital cost; inference is a small cost paid forever. That shape is why the business model is per-token.

Figures for Llama 2 70B · Karpathy (2023)



What it actually does: guess the next chunk

One operation, repeated. That is the entire trick.



Then it feeds its own answer back in and does it again. And again. That is how you get a paragraph.

It is not retrieving. It is generating what is statistically likely to come next. Karpathy's word for it is “dreaming” — the model dreams documents that look like its training data. Sometimes the dream is true. It has no mechanism for telling the difference.

↳ The output is a probability distribution over the whole vocabulary at every step, not a single word. Which token you actually get depends on the sampling strategy — see appendix.

LIVE DEMO

Why it cannot count the r's in “strawberry”

The word never reaches the model as letters.

[screen share — Colab, tokenizer notebook]

This explains three of the eight cards you got wrong before the break.

Fallback on the next slide if the notebook misbehaves.



What the demo showed

Three failures, one mechanism.

Cannot count letters

The word is 3 tokens, not 10 letters. Counting means recalling a property of an opaque ID — there is no character index to walk.

Cannot hit an exact word count

Same root, plus no lookahead. It commits to each token without a plan for the total.

Cannot read a clock

Different mechanism, same shape. Vision models see image patches, not angles. (Appendix.)

It writes an excellent sonnet — fourteen lines, rhyme scheme, meter — because form is style. Length is arithmetic.

Byte-pair encoding merges frequent character sequences into single tokens. Spelling knowledge is learned incidentally from wordplay and hyphenation in training text — never systematically. Tool use or chain-of-thought spelling both fix it, which is why reasoning models often now pass.



How a text predictor becomes an assistant

Three stages of training. Only the first is expensive.

1**PRETRAINING***10 TB of internet text**Millions of dollars. Done rarely.***Makes it KNOWLEDGEABLE.**

What comes out is a base model. It only continues documents — ask it a question and it may reply with more questions, because that is what a page of questions looks like.

2**FINE-TUNING***~100,000 written conversations**Cheap. Done often.***Makes it USEFUL.**

Humans write ideal question-and-answer pairs. It learns the shape of being asked something and answering. It is not learning new facts here — it is learning a format.

3**PREFERENCE TUNING***Humans ranking answers**Cheap. Continuous.***Makes it AGREEABLE — and makes it refuse.****→ Class 6**

Ranking two answers is far easier than writing a perfect one, so you can gather a lot of it. This is where politeness comes from. It is also where refusals come from.

10 TB versus 100,000 conversations — five orders of magnitude. Nearly everything it KNOWS came from stage 1. Nearly everything about HOW IT BEHAVES came from 2 and 3.



There is a fourth option — and it is the one you will use

Configuration. No training of any kind.

TRAINING

Stages 1–3

You change the WEIGHTS.

- Needs data, compute and expertise
- Takes days or weeks
- Permanent — you have made a new model
- A handful of organisations do stage 1

CONFIGURATION

What you will do

You change the INPUT.

- A system prompt telling it how to behave
- Documents it can search
- Optionally, tools it may call
- Free, instant, reversible, no data science

This is what a “custom GPT” is. Nothing is trained. In the next hour you will meet a company whose employees built 1,400 of them in nineteen months — precisely because the barrier was writing instructions, not training models.

A1 Parts 2 and 3 are configuration. You will not train anything in this course.



Fine-tuning redirects the dream. **It does not stop the dreaming.**

A fine-tuned model hallucinates just as readily as a base one. It has simply learned to hallucinate in the shape of a helpful assistant.

So: trust output you can verify. Treat everything else as a confident guess.

So — why is the frontier jagged?

Two mechanisms. You have now seen both of them happen.

REPRESENTATION

The model only sees what the encoding left behind.

- Text becomes tokens. “strawberry” arrives as three ID numbers — there are no letters anywhere in the system.
- Images become patches. The fine angle between two clock hands is blurred by the encoding.
- Ask about information the encoding destroyed, and it can only guess.

PREDICTABLE — know the encoding, know the failure.

EMERGENCE

Capabilities arrive with scale, unannounced.

- Multi-step arithmetic, reasoning step by step, learning from examples inside the prompt — near-useless at small scale, present at large.
- Nobody could say in advance which would appear, or at what size.
- The only way to find out what a model can do is to try it.

NOT PREDICTABLE — there is no rule. Only testing.

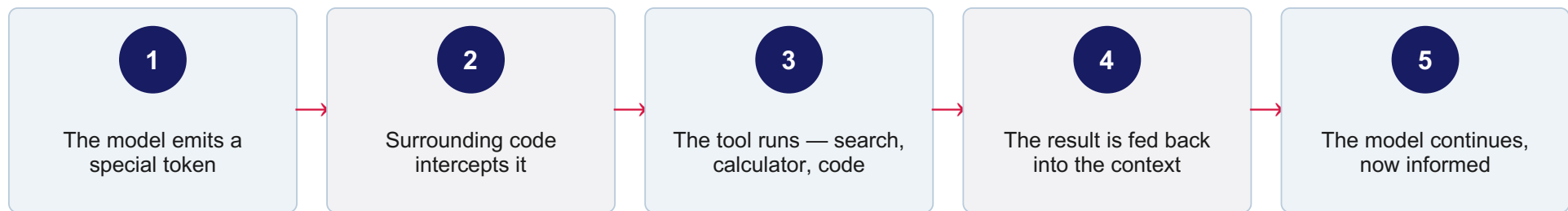
One half you can reason about. The other half you cannot — so you test it. That is what A1 is for.

Neither is solved by scale alone. The first needs a different encoding, or a tool that can count. The second does not get more predictable as models grow — if anything, less.



From talking to doing

An agent is not a new kind of model. It is a model, plus tools, plus a loop.



It did not “decide” to use a tool. Fine-tuning taught it when to emit that token, by example. Everything agentic you have read about is this loop, wrapped in better engineering.

↳ The token is a structured function call the runtime parses and executes. Nothing is autonomous — the loop is ordinary software calling a model repeatedly. Errors compound across steps, which is why multi-step agents fail far more than single calls.

What agents can and cannot do — right now

12% → 66.3%

Agent success on real computer tasks, in one year

OSWorld benchmark

1 in 3

Attempts still fail on structured benchmarks

Within 6 points of human — but not there

single digits

Share of business functions with agents deployed

Against 88% organisational AI adoption

The gap between the demo and the deployment is the story. It is also where your end-of-course project lives.

Source: Stanford HAI, AI Index Report 2026



One more thing: it is not only text

Multimodal — the same architecture, extended to images, audio and video.

What it means in practice

- A photograph of a whiteboard becomes working code
- A recording becomes minutes, then actions
- A chart becomes a paragraph explaining it

Why it matters more than it sounds

Most of the world's business data was never text. It is scans, photographs, recordings, forms, video. Text-only AI could not reach it.

Veo 3, tested across 18,000 generated videos, simulated buoyancy and solved mazes it was never trained on.

Awareness only today — the taxonomy needs it. We do not build with it in this course.

Source: Stanford HAI, AI Index Report 2026

POLL · CHECK THE MECHANISM

Why can't a model count the letters in “strawberry”?

- A It was never trained on spelling
- B It sees tokens, not letters — there is no character index
- C The word is too rare in training data
- D It can, but it chooses not to bother

Wooclap results

There is a correct answer to this one. Wooclap reveals it when the vote closes.



You now know what these things are,
how they are built, and where they break.

Next: what happens when a company hands them to six thousand people — and lets everyone build whatever they want.

A real company. A real decision. Still unresolved.



APPENDIX

Three slides of mechanism,
for anyone who wants the real thing.



Inside the model: attention and the context window

What attention actually does

At every position the model computes how much each earlier token should influence the next prediction. Those weights are learned, not fixed — which is how it handles “the animal didn’t cross the street because it was too tired” and knows what “it” refers to. Before transformers, that relationship had to travel step by step through a chain; attention lets any token look directly at any other in one operation.

Why the context window is a hard wall

Attention compares every token with every other, so cost grows roughly with the square of the length. Double the context, quadruple the work. That is why context limits exist, why long documents cost disproportionately more, and why retrieval — fetching only the relevant passages — is usually cheaper than pasting everything in.

What the parameters are not

There is no database inside. No lookup table, no facts stored in rows. Knowledge is distributed across billions of weights in a form nobody can read directly. This is why you cannot patch a specific wrong fact, and why interpretability is an open research field rather than an engineering task.



Why the same prompt gives different answers

The most common confusion about these systems, and it has a simple cause.

The model does not output a word. It outputs a probability for every token in its vocabulary — often 100,000 of them.

Something then has to pick one. Always take the highest-probability token and the output is repetitive and flat. Sample from the distribution instead and you get variety — and a different answer each time.

Temperature

Flattens or sharpens the distribution. Near 0 is near-deterministic and dull; higher is more varied and less reliable.

Top-p / top-k

Restricts sampling to the most likely candidates, cutting off the improbable tail that produces nonsense.

What this means for you

An eval run once is not evidence. If you change a prompt and the output improves, that may be sampling, not your prompt. Run it repeatedly — which is exactly what A1 Part 3 asks.



Training is a one-off. Inference is forever.

The cost shape that determines almost every deployment decision.

TRAINING

One-time capital cost

\$2M for a mid-size model, far more at the frontier. Paid once, by someone else, before you ever touch it. Irrelevant to your unit economics.

INFERENCE

Per-call operating cost

Small individually. Paid on every single call, forever, and it scales linearly with usage. This is the number that decides whether your product works.

This is why the industry prices per token, why a classifier can be cheaper than an API at volume even after training cost, and why “it works in the demo” tells you nothing about whether it works at a million users. Class 5 does this properly.

